NSS COLLEGE OF ENGINEERING, PALAKKAD - 678008.

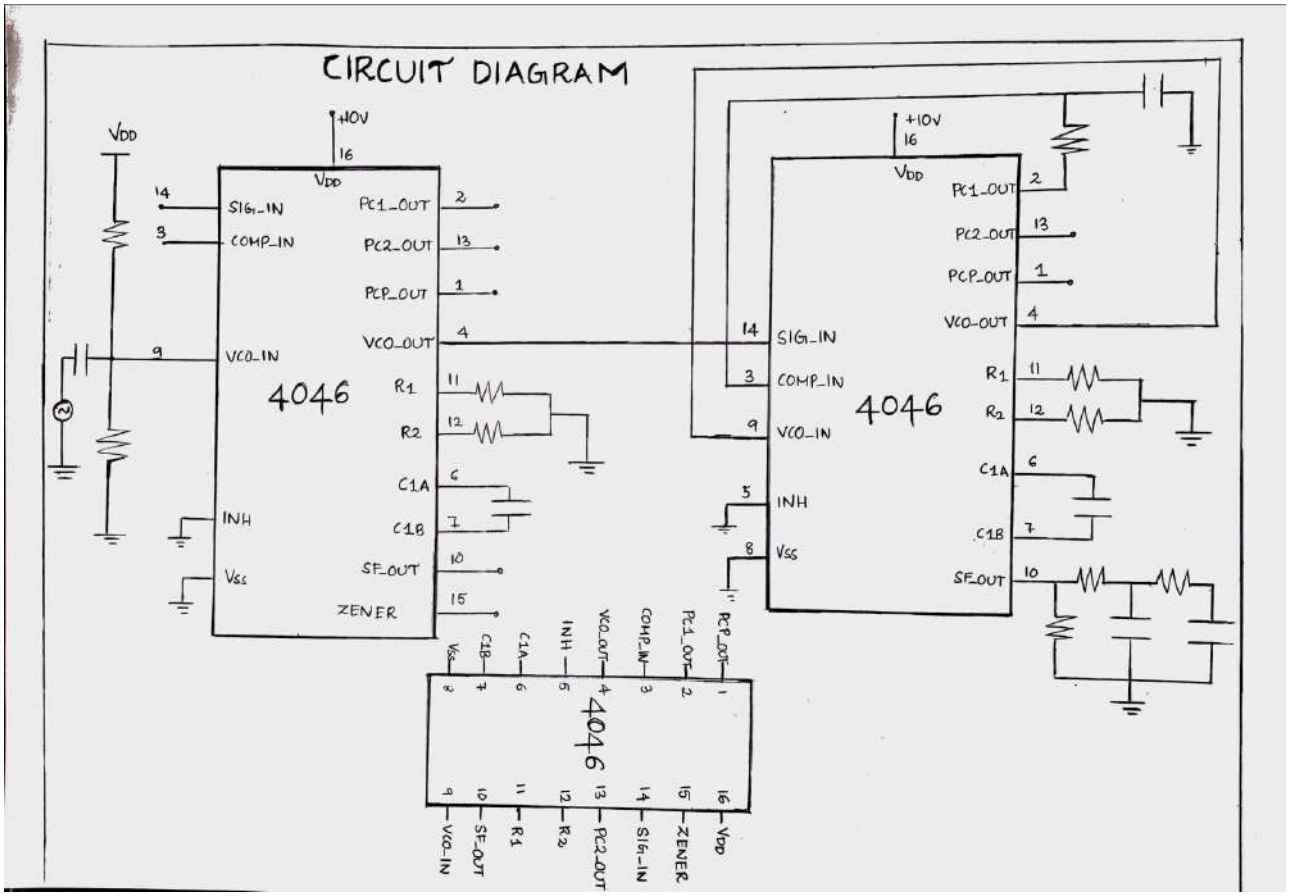
ECL 332 - COMMUNICATION LAB

TABLE OF CONTENTS

Part A

HARDWARE EXPERIMENTS

CIRCUIT DIAGRAM



EXPERIMENT - 1

FREQUENCY MODULATION AND DEMODULATION

AIM

To design and setup a frequency modulator and demodulator circuit using PLL IC CD4046.

COMPONENTS REQUIRED

IC CD 4046 - 2 Nos.

Resistors: 10kΩ – 8 Nos., 100Ω - 2 Nos.

Capacitors: 1μF - 1 No., 0.02μF – 2 Nos., 0.01μF - 3 Nos.

Function Generator, DC Power Supply , Digital Storage Oscilloscope(DSO) , Breadboard and Multimeter.

THEORY

Frequency modulation is a technique or a process of encoding information on a particular signal (analogue or digital) by varying the carrier wave frequency in accordance with the frequency of the modulating signal.

FM(t) = fc + k Am cos (2π fm t)

FM(t) = fc + k m(t)

where,

FM(t) is frequency modulated wave,fc is frequency of the carrier wave

m(t) is modulating signal,k is proportionality constant

Frequency demodulation is the inverse of frequency modulation. The original modulating signal is obtained as output following demodulation. After the signal has been received, filtered, and amplified, the original modulation from the carrier must be recovered. This is known as demodulation or detection.

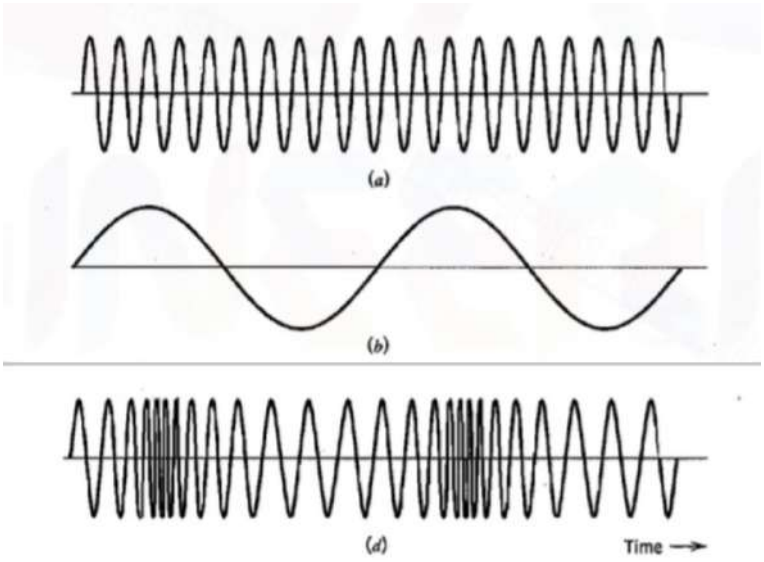


Figure 1: (a)carrier signal, (b) message signal, (c) FM signal

PROCEDURE

1). Test all the components and probes.

2). Setup the circuit on the breadboard.

3). Feed 5 $V_{P\text{-}P}$ 1kHz square wave input.

4). Observe the output waveform in the DSO.

5). Plot the waveforms.

DESIGN

Supply V_{cc} = 5V at Pin 16 and ground Pin 8 both PLL ICs.

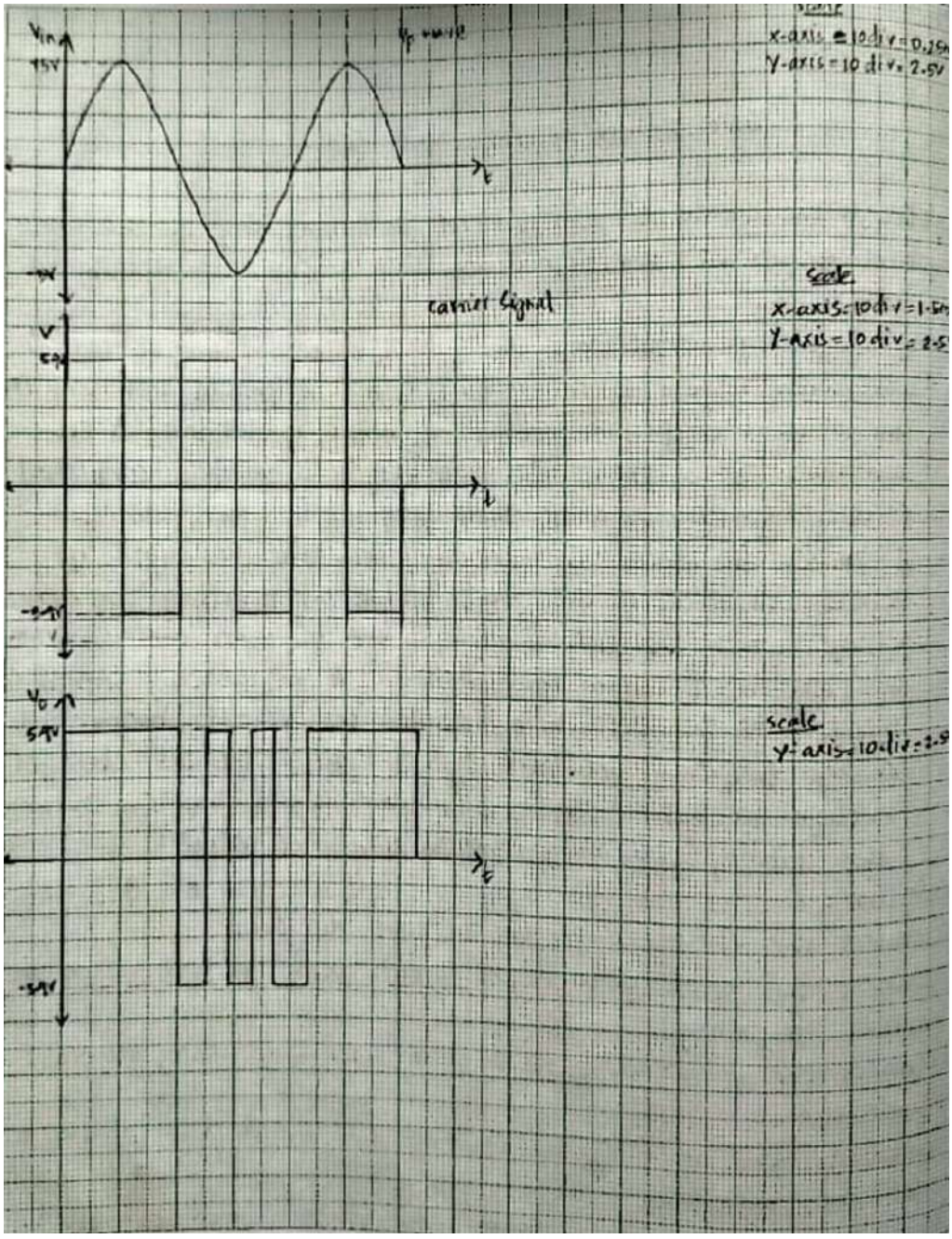
MODULATOR

Use a voltage divider network of 2 resistors with R = 10kΩ clamping the control input (Pin 9) at $\frac{V_{cc}}{2} = 2.5V$. Give a message signal of frequency 1kHz and amplitude $1V_{P\text{-}P}$ and control voltage input (Pin 9) through capacitor of C1 = 1μF select R1 = 10kΩ (Pin 11) , R2 = 10kΩ(Pin 12), C = 0.02 μF between (Pin 6-Pin 7) so our free running frequency(f_o) is of the form:-

$$f_o = \frac{0.16 \times 2.5}{10 \times 10^3 \times 0.002 \times 10^{-6}} + \frac{1}{100 \times 10^3 \times 0.002 \times 10^{-6}} = 20kHz + 5kHz = 25kHz$$

The FM output is obtained from V_{cout} (Pin 4) of PLL IC (PLL 1).

OBSERVATIONS



DEMODULATOR

Use the same R1,R2 resistors for the second PLL IC(PLL 2) so that the free running frequency remain the same as that of the modulating frequency. Feed the signal input pin of phase detector (Pin 14) The second IC with the FM signal. The other input of phase detector (Pin 3) is fed with V_{co} output (Pin 4) The output of phase detector (Pin 9) through a low pass filter with R3 = 10kΩ and C1 = 0.01μF. The demodulated output is obtained from the Pin 10 by pulling down using a resistor R_p = 10k$Omega$. It is then low pass filtered at cut off frequency, f_c=1.5kHz to eliminate higher order ripples.

$$f_c = \frac{1}{2\pi \times R_f C_f} = 1.5kHz$$

Choose C_f = 0.001μF, therefore R_f = 10kΩ.

RESULT

Frequency modulator and demodulator circuit using PLL has been implemented using IC CD 4046 successfully.

CIRCUIT DIAGRAM

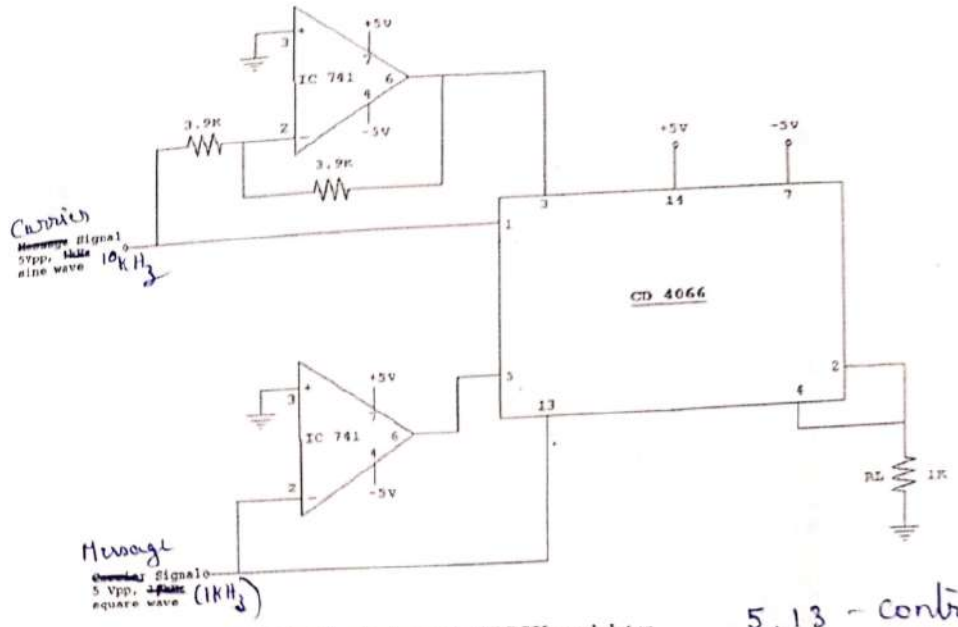


Figure 2.3: Circuit diagram of BPSK modulator

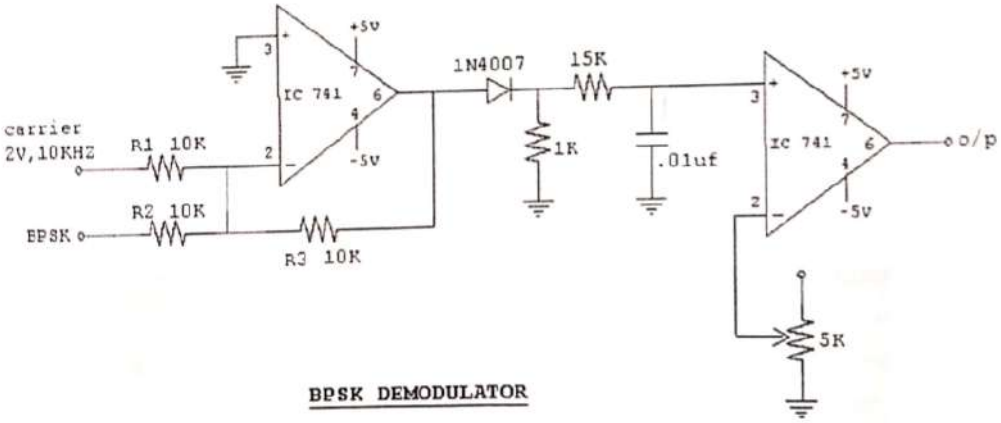


Figure 2.4: Circuit diagram of BPSK demodulator

EXPERIMENT - 2

BINARY PHASE SHIFT KEYING(BPSK) MODULATOR AND DEMODULATOR

AIM

To setup a Binary Phase Shift Keying(BPSK) modulator and demodulator circuit and observe the waveforms.

COMPONENTS REQUIRED

IC LM741 - 4 Nos.

IC CD4066 - 1Nos.

Resistors: 3.9kΩ – 2 Nos.,1kΩ – 1 No.,10kΩ – 2 Nos., 15KΩ - 1 No.

Capacitors: 0.01μF - 1 No.

Function Generator, DC Power Supply , Digital Storage Oscilloscope(DSO) , Breadboard and Multimeter.

THEORY

In a coherent binary PSK system, the pair of signals $s_1(t)$ and $s_2(t)$ used to represent binary symbols 1 and 0, respectively, is defined by

$$S_1(t) = \sqrt{\frac{2E_\mathrm{b}}{T_\mathrm{b}}} \cos 2\pi f_\mathrm{c} t$$

$$S_2(t) = \sqrt{\frac{2E_\mathrm{b}}{T_\mathrm{b}}} \cos 2\pi f_\mathrm{c} t + \pi = -\sqrt{\frac{2E_\mathrm{b}}{T_\mathrm{b}}} \cos 2\pi f_\mathrm{c} t$$

where $0 <= t <= T_0$ and $E_\mathrm{b}$is the transmitted signal energy per bit. To ensure that each transmitted bit contains an integral number of cycles of the carrier wave, the carrier frequency f_c is chosen equal to $n_\mathrm{c}/T_\mathrm{b}$, for some fixed integer n_c. A pair of sinusoidal waves that differ only in a relative phase-shift of 180 degrees, as defined above, are referred to as antipodal signals. In the case of binary PSK, there is only one basis function of unit energy, namely, $\Phi_1(t) = \sqrt{\frac{2E}{T_\mathrm{b}}} \cos 2\pi f_c t$
Then we may express the transmitted signals $s_1(t)$and $s_2(t)$in terms of $\Phi_1(t)$ as follows:

$$s_1(t) = \sqrt{E_\mathrm{b}}\Phi_1(t)$$

$$s_2(t) = -\sqrt{E_\mathrm{b}}\Phi_1(t)$$

MODULATOR: This circuit mainly consists of a unity gain operational amplifier and 2 switches. Inverting amplifier provide 180 degrees phase shift to carrier signal. The binary symbols are applied to the control input of the two switches. An operational amplifier inverting zero crossing detector enables to apply symbol 1 at the control input of one switch while symbol 0 at the control input of the other switch. BPSK Signal has constant amplitude as in the case of FPSK Signal. Therefore noise can be removed easily.

DEMODULATOR: BPSK Signal can be demodulated non-coherently by converting it into a BASK signal and then by using an envelope detector. The original carrier is added with BPSK Signal using a summing amplifier to obtain a BASK Signal which is further demodulated using an envelope detector. The output of envelope detector may not be square-shaped. So it is converted to square wave using comparator.

PROCEDURE

1). Test all the components and probes.

2). Setup the modulator circuit on the breadboard.

3). Give the Polar NRZ level encoded binary message signal to Pin 13 and its inverted version to Pin 5 of IC4066.

4). Give 2VP-P 10 kHz sinusoidal signal as the carrier signal to Pin1 and its inverted version to Pin 3 of CD4066.

4). Observe the BPSK output.

5). Setup the Demodulator circuit on breadboard.

6). Give the BPSK signal and the carrier signal as the input of the demodulator circuit.

7). Compare the reconstructed output with the original binary message signal.

DESIGN

For the modulator,

Gain at inverting amplifier A = $\frac{-Rf}{Ri}$

$\therefore Rf = Ri = 3.9k\Omega$

For the demodulator,

Take R1 = R2 = R3 = 10kΩ and the rectifier resistor R_{rect} = 1kΩ.

$$f = \frac{1}{T} = \frac{1}{2\pi \times R \times C} = 1kHz,$$

OBSERVATIONS

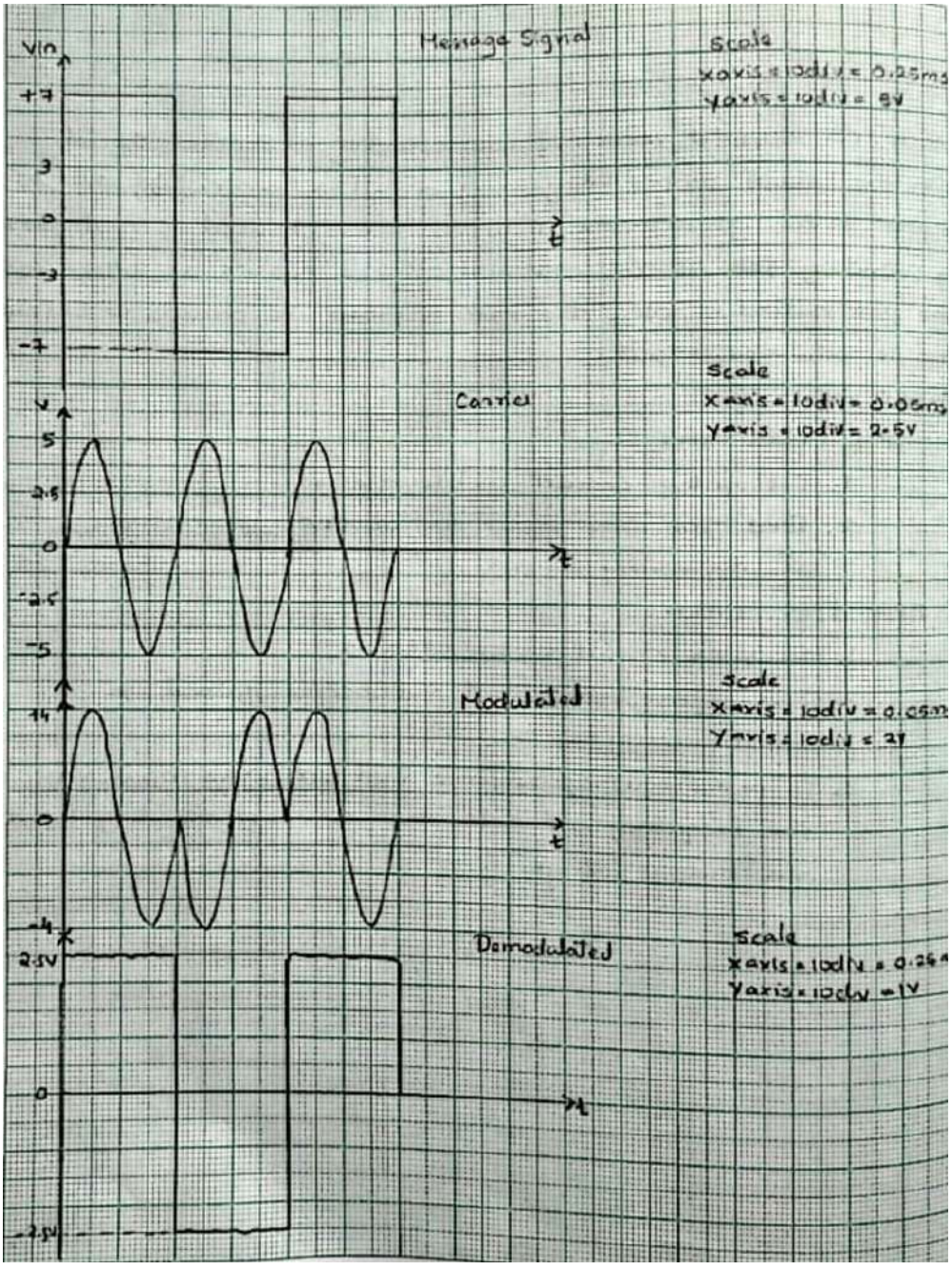


Figure 2: Results

take C = 0.01μF.

$$\therefore R = 15k\Omega$$

Use 10kΩ Potentiometer to set the threshold voltage for the comparators.

RESULT

BPSK modulator and demodulator has been implemented successfully and results have been verified.

Part B

SIMULATION EXPERIMENTS

EXPERIMENT - 1

ERROR PERFORMANCE OF BINARY PHASE SHIFT KEYING(BPSK)

AIM:

To verify the error perfromance of Binary Phase Shift Keying.

THEORY:

Binary Phase Shift Keying is the simplest form of phase shift leying. Also called the reversal phase shift keying or 2PSK, it uses two phases that are seperated by 180 degree.

The generation formula for BPSK is,

$$S_1(t) = \sqrt{\frac{2E_\mathrm{b}}{T_\mathrm{b}}} \cos 2\pi f_\mathrm{c} t$$

$$S_2(t) = \sqrt{\frac{2E_\mathrm{b}}{T_\mathrm{b}}} \cos 2\pi f_\mathrm{c} t + \pi = -\sqrt{\frac{2E_\mathrm{b}}{T_\mathrm{b}}} \cos 2\pi f_\mathrm{c} t$$

Where 0<=t<=Tb is the transmitted signal energy per bit .The carrier frequency fc is choosen equal to nc/Tb for some integer nc . A part of sinusoidal signal that differ by a phase shift of 180° are caldle antipodal signals

The bit error rate (BER) can be calculated under AWGN channel as

$$P_\mathrm{c} = \frac{1}{2} erfc \sqrt{\frac{Eb}{N0}}$$

PYTHON CODE

```
import numpy as np
import matplotlib.pyplot as plt
from numpy import sum,sqrt
from numpy.random import standard_normal
from scipy.special import erfc
def modulator(ak,L):
    from scipy.signal import upfirdn
    a_bb=upfirdn(h=[1]*L,x=2*ak-1,up=L)
    t=np.arange(0,len(ak)*L)
    return (a_bb,t)
def demod(r_bb,L):
    x=np.real(r_bb)
    x=np.convolve(x,np.ones(L))
    x=x[L-1:-1:L]
    ak_hat=(x>0).transpose()
    return ak_hat
def awgn(S,SNR_db,L=1):
    gamma=10**(SNR_db/10)
    p=L*sum(abs(S)**2)/len(S)
    N0=p/gamma
    n=sqrt(N0/2)*standard_normal(S.shape)
    r=S+n
    return r
N=int(input('Enter the number of samples'))
L=16
fc=800
Eb_N0=np.arange(-4,11,2)
fs=L*fc
bit=np.random.randint(0,2,N)
(msg,t)=modulator(bit,L)
plt.plot(t,msg)
```

```
plt.title('message signal')
plt.show()
c=np.cos(2*np.pi*fc*t/fs)
plt.plot(t,msg*c)
plt.title('modulated signal')
plt.show()
BER=np.zeros(len(Eb_N0))
for i,EbN0 in enumerate(Eb_N0):
    r=awgn(msg*c,EbN0,L)
    r_bb=r*np.cos(2*np.pi*fc*t/fs)
    ak_hat=demod(r_bb,L)
    BER[i]=np.sum(bit!=ak_hat)/N
plt.plot(np.real(r_bb[0:100]),np.imag(r_bb[0:100]),marker='o')
plt.title('constellation of bpsk')
plt.show()
plt.plot(t,r)
plt.title('demodulated signal')
plt.show()
TheoreticalBER=0.5*erfc(np.sqrt(10**(Eb_N0/10)))
plt.semilogy(Eb_N0,BER,'k*',label='simulated BER')
plt.semilogy(Eb_N0,TheoreticalBER,'r--',label='Theoretical BER')
plt.xlabel('Eb/N0')
plt.ylabel('probability of Bit error')
plt.title('SNR vs BER in BPSK (N=10000)')
plt.show()
```

1 ALGORITHM

The code you provided is an implementation of a Binary Phase Shift Keying (BPSK) modulation and demodulation system with Additive White Gaussian Noise (AWGN). Here's a step-by-step algorithm for the given code:

1. Import Required Libraries: - Import 'numpy', 'matplotlib.pyplot', and necessary functions from 'numpy.random' and 'scipy.special'.

2. Define Functions: 'modulator(ak, L)': Modulates the input binary sequence 'ak' with an upsampling factor 'L'.

'demod($r_b b, L$)' : $Demodulatesthereceivedbasebandsignal`r_b b`withanupsamplingfactor`L`$.

'awgn(S, $SNR_d b, L = 1$)' : $AddsAWGNtothesignal`S`foragivenSignal-to-NoiseRatio(SNR)indB$.

3. User Input: - Prompt the user to input the number of samples 'N'.

4. Parameters Initialization: - Set the upsampling factor 'L', carrier frequency 'fc', and calculate the sampling frequency 'fs'. - Define a range for 'Eb/N0' values.

5. Generate Random Bit Sequence: - Generate a random binary sequence of length 'N'.

6.Modulate the Bit Sequence: - Call the 'modulator' function to get the modulated message signal and the corresponding time vector. - Plot the message signal.

7. Modulate the Signal with Carrier: - Modulate the baseband signal with a cosine carrier. - Plot the modulated signal.

8. Initialize BER Array: - Initialize an array to store Bit Error Rates (BER) for different 'Eb/N0' values.

9. Simulate Transmission Over AWGN Channel: - For each 'Eb/N0' value: 1. Add AWGN to the modulated signal. 2. Demodulate the received signal. 3. Calculate the BER by comparing the original bit sequence with the demodulated sequence.

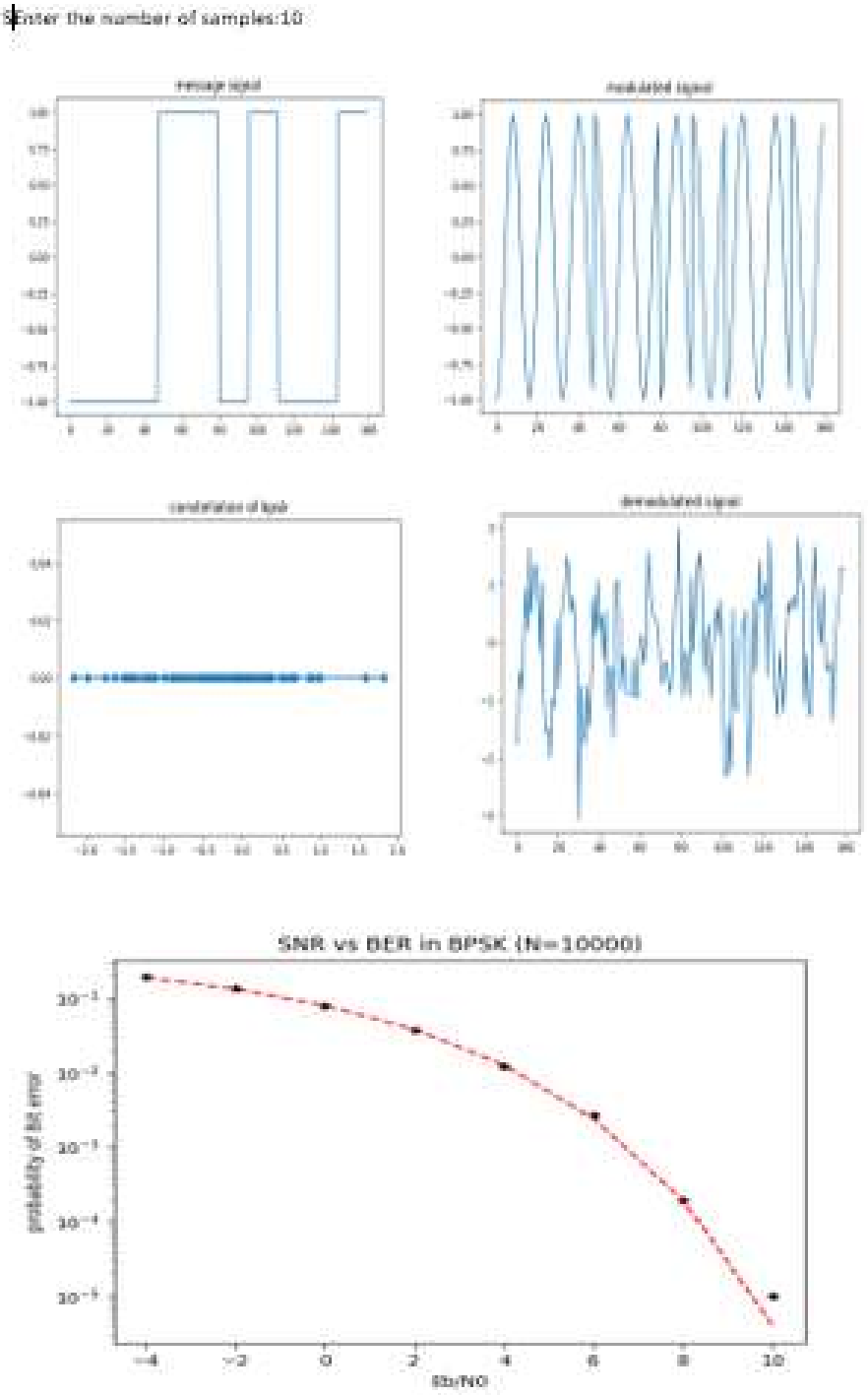
10. Plot Constellation Diagram: - Plot the real and imaginary parts of the received baseband signal to visualize the BPSK constellation.

11. Plot Demodulated Signal: - Plot the demodulated signal.

12. Calculate and Plot Theoretical BER: - Calculate the theoretical BER using the 'erfc' function. - Plot the simulated BER and theoretical BER on a semi-logarithmic scale.

This algorithm outlines each step in the code, from importing libraries to plotting the results.

OBSERVATIONS



RESULTS:

BPSK Error Performance curve was plotted successfully.

EXPERIMENT - 2

ERROR PERFORMANCE OF QUADRATURE PHASE SHIFT KEYING(QPSK)

AIM:

To verify the error performance of Quadrature Phase Shift Keying.

THEORY:

Quadrature phase shift keying is an extension of BPSK wherein 2 bits are modulated at once and they reduce the carrier frequency signalling and bandwidth,

In QPSK binary symbol is represented by a set of bits called dibits (00,01,10,11) and the phase of the carrier takes values $\pi/4, 3\pi/4, 5\pi/4, 7\pi/4$,

The transmitted QPSK signal is given by the expression

$$s_1(t) = \sqrt{\frac{2E_\mathrm{b}}{T_\mathrm{b}}} \cos 2\pi f_\mathrm{c} t + (2i-1)\frac{\pi}{4}$$

There are 2 orthonormal basis functions ϕ1(t) and ϕ2(t) defined by a pair of quadrature carrier

$$\phi_1(t) = \sqrt{\frac{2}{T_\mathrm{b}}} \cos 2\pi f_\mathrm{c} t$$

$$\phi_2(t) = \sqrt{\frac{2}{T_\mathrm{b}}} \cos 2\pi f_\mathrm{c} t$$

In QPSK two successive bits are combined . When a symbol is changed to the next symbol . The phase of the carrier is changed to 45°. Although QPSK can be viewed as quarternary modulation it is easier to see it as a independently modulated quadrature channel. With this interpolation the even (or odd) bit are used to modulate the inphase component of the carrier while the odd (or even) bits are used to modulate the Quadrature channel . With this interpolation the even (or odd) but are used to modulate the Quadrature component BPSK is used to on both carrier and they can be independently demodulated . As a result, the BER of QPSK is the same as of BPSK and is given by

$$P_\mathrm{c} = \frac{1}{2} erfc \sqrt{\frac{Eb}{N0}}$$

Advantage of coherent QPSK is that for the same BER it has reduced bandwidth compared to BPSK. Also it's information transmission rate is higher.

PYTHON CODE

```
import numpy as np
import matplotlib.pyplot as plt
from numpy import sum,sqrt
from numpy.random import standard_normal
from scipy.signal import upfirdn
from scipy.special import erfc
def qpsk_mod(a,fc,of):
    L=2*of
    I=a[0::2];Q=a[1::2]
    I=upfirdn(h=[1]*L,x=2*I-1,up=L)
    Q=upfirdn(h=[1]*L,x=2*Q-1,up=L)
    plt.plot(I)
    plt.title('inphase msg signal')
    plt.show()
    plt.plot(Q)
    plt.title('quadphase msg signal')
    plt.show()
    fs=of*fc
    t=np.arange(0,len(I)/fs,1/fs)
    I_t=I*np.cos(2*np.pi*fc*t)
    Q_t=-Q*np.sin(2*np.pi*fc*t)
    plt.plot(t,I_t)
    plt.title('inphase modulated signal')
    plt.show()
    plt.plot(t,Q_t)
    plt.title('quadphase modulated signal')
    plt.show()
    s_t=I_t+Q_t
    plt.plot(s_t)
    plt.title('transmitted signal')
    plt.show()
```

```
    return s_t,I,Q


def awgn(s_t,snrdb,of):
    g=10**(snrdb/10)
    p=of*sum(abs(s_t)**2)/len(s_t)
    N0=p/g
    n=sqrt(N0/2)*(standard_normal(s_t.shape))
    r=s_t+n
    return r

def qpsk_demod(r,fc,of):
    fs=of*fc
    L=2*of
    t=np.arange(0,len(r)/fs,1/fs)
    x=r*np.cos(2*np.pi*fc*t)
    y=-r*np.sin(2*np.pi*fc*t)
    x=np.convolve(x,np.ones(L))
    y=np.convolve(y,np.ones(L))
    x=x[L-1::L]
    y=y[L-1::L]
    a_hat=np.zeros((2*len(x)))
    a_hat[0::2]=(x>0)
    a_hat[1::2]=(y>0)
    return(a_hat,x,y)

N=30
a=np.random.randint(2,size=N)
EbNodb=np.arange(-4,11,2)
of=8
fc=800
```

```
BER=np.zeros(len(EbNodb))
s_t,i,q=qpsk_mod(a,fc,of)
plt.scatter(i,q)
plt.title("Signal constellation")
plt.show()
for i,EbNO in enumerate(EbNodb):
    r=awgn(s_t,EbNO,of)
    a_hat,x,y=qpsk_demod(r,fc,of)
    BER[i]=np.sum(a!=a_hat)/N

plt.plot(x[0:200],y[0:200],"o")
plt.show()
tber=0.5*erfc(np.sqrt(10**(EbNodb/10)))
plt.semilogy(EbNodb,BER,'ko',label='simulated')
plt.semilogy(EbNodb,tber,'r-',label='theoretical')
plt.xlabel("Eb/No")
plt.ylabel("Probability of bit error")
plt.title("SNR vs BER in QPSK for N=100000")
plt.show()
```

2 ALGORITHM

The code you provided implements a Quadrature Phase Shift Keying (QPSK) modulation and demodulation system with Additive White Gaussian Noise (AWGN). Here's a step-by-step algorithm for the given code:

1. Import Required Libraries:

- Import 'matplotlib.pyplot', 'numpy' functions, and necessary functions from 'numpy.random', 'scipy.signal', and 'scipy.special'.

2. Define Functions:

- qpsk$_m od(a, fc, of)$' : $Modulates the input binary sequence 'a' with carrier frequency 'fc' and oversampling factor 'o$

- 'awgn(s$_t, snrdb, of)$' : $Adds AWGN to the signal 's_t' for a given Signal - to - Noise Ratio (SNR) in dB.$

- 'qpsk$_d emod(r, fc, of)$' : $Demodulates the received signal 'r' with carrier frequency 'fc' and oversampling factor 'of$

3. QPSK Modulation:

- Split the input binary sequence 'a' into in-phase (I) and quadrature (Q) components.

- Upsample the I and Q components.

- Plot the I and Q message signals.

- Modulate the I and Q components with cosine and sine carriers, respectively.

- Plot the in-phase and quadrature modulated signals.

- Combine the modulated I and Q components to form the transmitted signal 's$_t$'.

- Plot the transmitted signal.

4. Add AWGN:

- Define 'awgn' function to add noise to the transmitted signal 's$_t$'.

5. QPSK Demodulation:

- Multiply the received signal 'r' with cosine and sine carriers to demodulate the I and Q components.

- Low-pass filter the demodulated signals.

- Downsample the filtered signals.

- Combine the demodulated I and Q components to reconstruct the binary sequence 'a$_h at$'.

6. Simulation Parameters:

- Set the number of samples 'N'.

- Generate a random binary sequence of length 'N'.

- Define a range for 'Eb/N0' values.

- Set the oversampling factor ‘of‘ and carrier frequency ‘fc‘.

- Initialize an array to store Bit Error Rates (BER) for different ‘Eb/N0‘ values.

7. Modulate and Plot Signal Constellation:

- Call the ‘qpsk$_{m}od‘functiontomodulatethebinarysequence‘a‘$.

- Plot the signal constellation by scattering the I and Q components.

8. Simulate Transmission Over AWGN Channel:

- For each ‘Eb/N0‘ value:

i. Add AWGN to the transmitted signal.

ii. Demodulate the received signal.

iii. Calculate the BER by comparing the original bit sequence with the demodulated sequence.

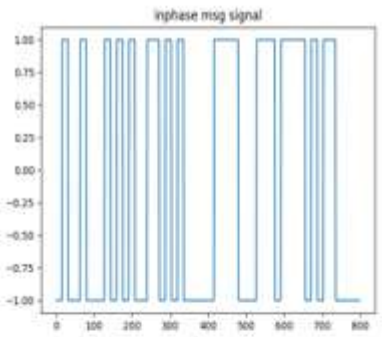
9. Plot BER and Theoretical BER:

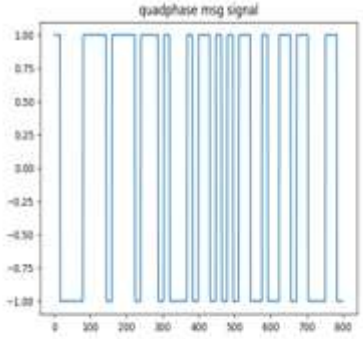
Plot the real and imaginary parts of the received baseband signal to visualize the QPSK constellation.

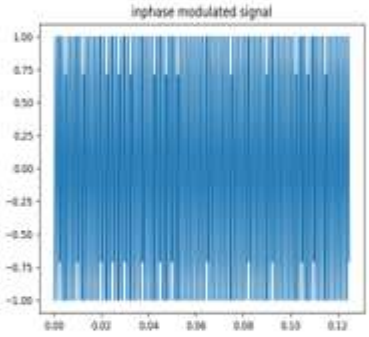
- Calculate the theoretical BER using the ‘erfc‘ function.

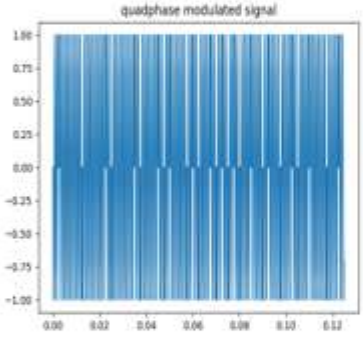
- Plot the simulated BER and theoretical BER on a semi-logarithmic scale.

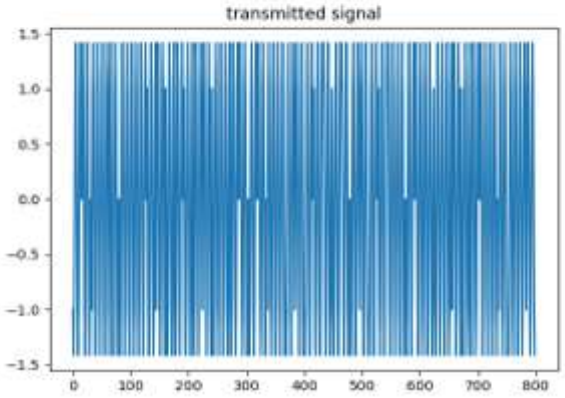
OBSERVATIONS

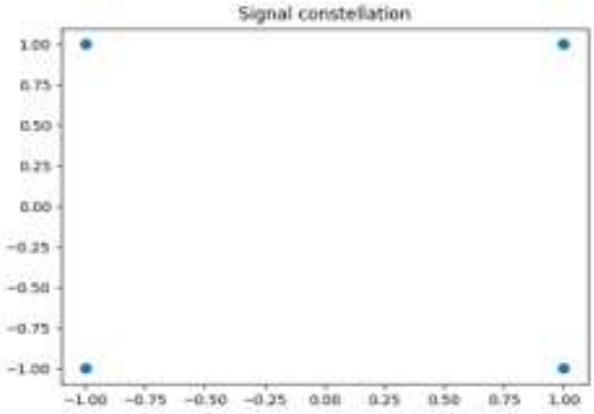


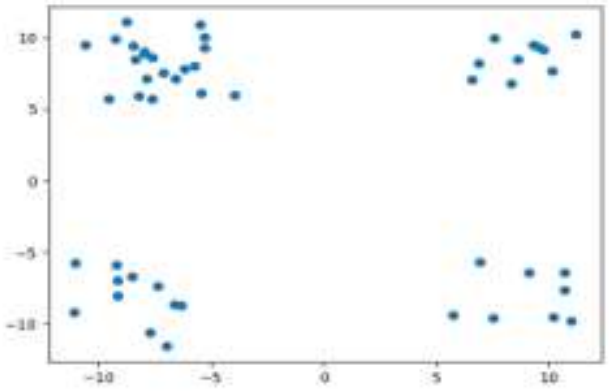


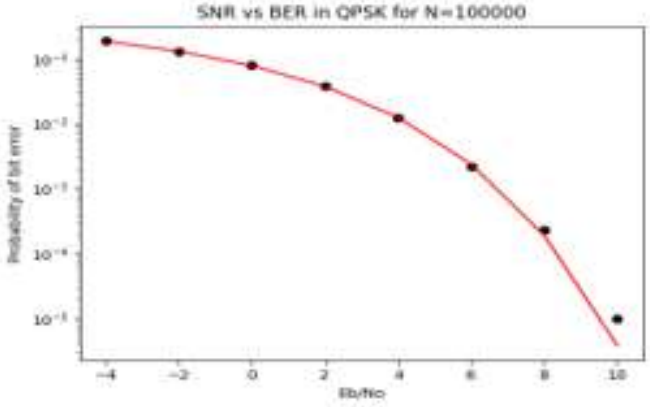







Signal constellation
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00 −0.75 −0.50 −0.25 0.00 0.25 0.50 0.75 1.00


10
5
0
−5
−10
−10 −5 0 5 10


SNR vs BER in QPSK for N=100000
Probability of bit error
Eb/No
−4 −2 0 2 4 6 8 10

RESULT:

QPSK Error Performance curve was plotted successfully.

EXPERIMENT - 3

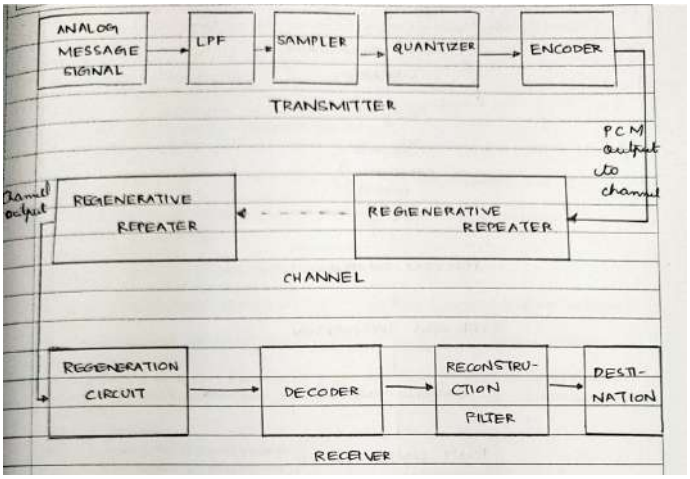
PERFORMANCE OF WAVEFORM CODING USING PULSE CODE MODULATION(PCM)

AIM:

To verify the performance of waveform coding using pulse code modulation.

THEORY:

Modulation is the process of varying one or more parameter of the carrier signal with instantaneous values of the message sigbal . There are many modulation technique used in PCM . A signal is said to be pulse code modulated when it converts it's analog information into binary sequence with respect to instantaneous values of a given sine wave . The transmitter section of PCM circuit is made of sampling , quantising and encoding which are performed in the analog to digital converter section. The low pass filter prior to sampling prevents aliasing of the message signal .The basic operators in the recieving section are regeneration of impared signals .decoding and reconstruction of quantised pulse train . The following block diagram of PCM which represent the basic elements of both the transmitter and receiver sections:-



PYTHON CODE

```
import numpy as np
import matplotlib.pyplot as plt


def quan(dc_shifted, level):
    range = max(dc_shifted) - min(dc_shifted)
    step_size = range / (2**level - 1)  # Scale the step size
    quantized_wave = np.round(dc_shifted / step_size) * step_size  # Quantize and then rescale
```

```
    return quantized_wave

def decimal_to_binary(decimal_value, num_bits):
    binary_str = bin(int(decimal_value))[2:].zfill(num_bits)
    return [int(bit) for bit in binary_str]

def calculate_power(signal):
    return np.mean(np.square(signal))

# Plotting the original wave
sampling_rate = 1000
duration = 1
f = 10
dc_offset = 2
t = np.linspace(0, duration, sampling_rate)
original_wave = np.sin(2 * np.pi * f * t)

# Shifting the original sine wave
dc_shifted = original_wave + dc_offset

# Plotting the waveforms
plt.subplot(3, 1, 1)
plt.plot(t, original_wave)
plt.title("Original Waveform")
plt.subplot(3, 1, 2)
plt.plot(t, dc_shifted)
plt.title("DC Shifted Waveform")
plt.subplot(3, 1, 3)
plt.plot(t, quan(dc_shifted, 8))
plt.title("Quantized Waveform")
plt.tight_layout()
plt.show()
```

```
quantized_wave = quan(dc_shifted, 8)

# Determine the maximum number of bits needed
max_bits = len(bin(int(max(quantized_wave)))) - 2
# Convert each quantized value to binary and pad with zeros
binary_encoded_wave = np.array([decimal_to_binary(value, max_bits) for value in quantized_wave
print("binary encoded wave=",binary_encoded_wave)

# Calculate power of shifted signal
signal_power = calculate_power(dc_shifted)

# Set up variables for plotting
levels = range(2, 17)  # Number of bits per symbol
SNRs = []

# Iterate over different levels (number of bits per symbol)
for level in levels:
    quantized_wave = quan(dc_shifted, level)
    quantization_error = dc_shifted - quantized_wave
    noise_power = calculate_power(quantization_error)
    # Compute SNR in dB
    SNR = 10 * np.log10(signal_power / noise_power)
    SNRs.append(SNR)

plt.plot(levels, SNRs)
plt.xlabel('Number of Bits per Symbol')
plt.ylabel('SNR (dB)')
plt.title('SNR versus Number of Bits per Symbol')
plt.show()
```

3 Algorithm

1.Import ‘numpy‘ for numerical operations and ‘matplotlib.pyplot‘ for plotting.

2. Define the ‘quan‘ function:

- Take ‘dc$_s hifted$‘$(signaltobequantized) and$‘$level$‘$(numberofquantizationlevels) asinputs.$

- Calculate the range of the signal.

- Compute the step size based on the number of quantization levels.

- Quantize the input signal by rounding to the nearest quantization level.

- Return the quantized waveform.

3. Define the ‘decimal$_t o_b inary$‘$function$:

- Take ‘decimal$_v alue$‘$(valuetobeconvertedtobinary) and$‘$num_b its$‘$(numberofbitsforthebinaryrepresentation) asin$

- Convert a decimal value to binary and pad with zeros to ensure the specified number of bits.

- Return the binary representation as a list of integers.

4. Define the ‘calculate$_p ower$‘$function$:

- Take ‘signal‘ (the signal for which power is to be calculated) as input.

- Compute the average power of the signal using the mean of the squared values.

- Return the power of the signal.

5. Generate parameters for the original waveform:

- Set sampling rate, duration, frequency, and DC offset.

- Generate time values using ‘numpy.linspace‘.

6. Plot the original sine wave and the DC shifted wave.

7. Quantize the DC shifted waveform using 8 bits and plot the quantized waveform.

8. Convert each quantized value to binary and pad with zeros to ensure a consistent number of bits.

9. Print the binary-encoded waveform.

10. Calculate the power of the DC shifted signal using the ‘calculate_power‘ function.

11. Iterate over different levels of quantization (2 to 16 bits):

- Compute the quantization error by subtracting the quantized waveform from the DC shifted waveform.

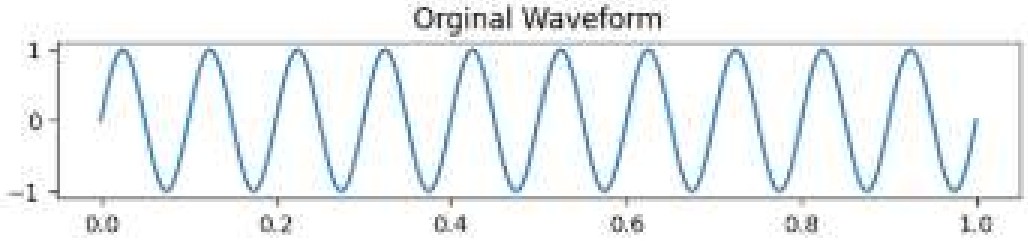
- Calculate the noise power from the quantization error.

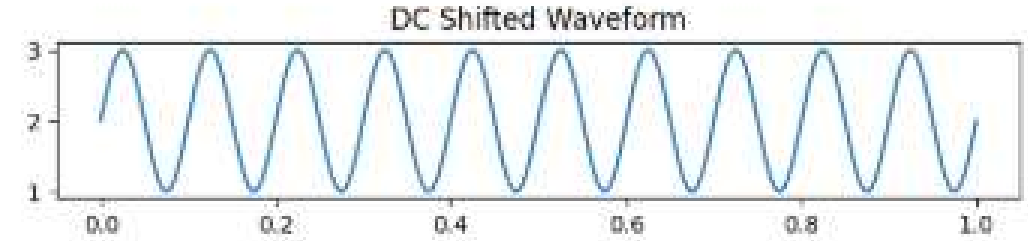
- Compute the SNR using the formula $\text{SNR} = 10 \cdot \log_{10}\left(\frac{\text{Signal Power}}{\text{Noise Power}}\right)$.

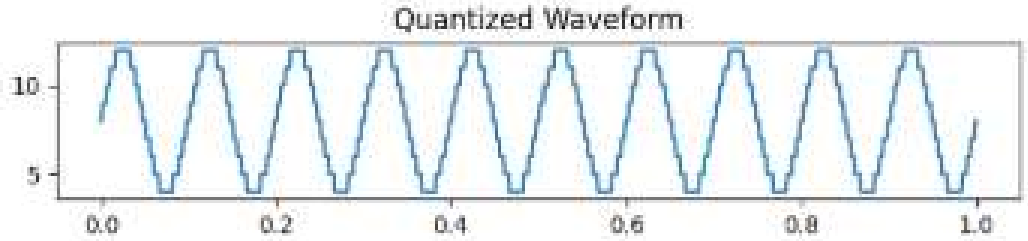
- Append the SNR values to a list.

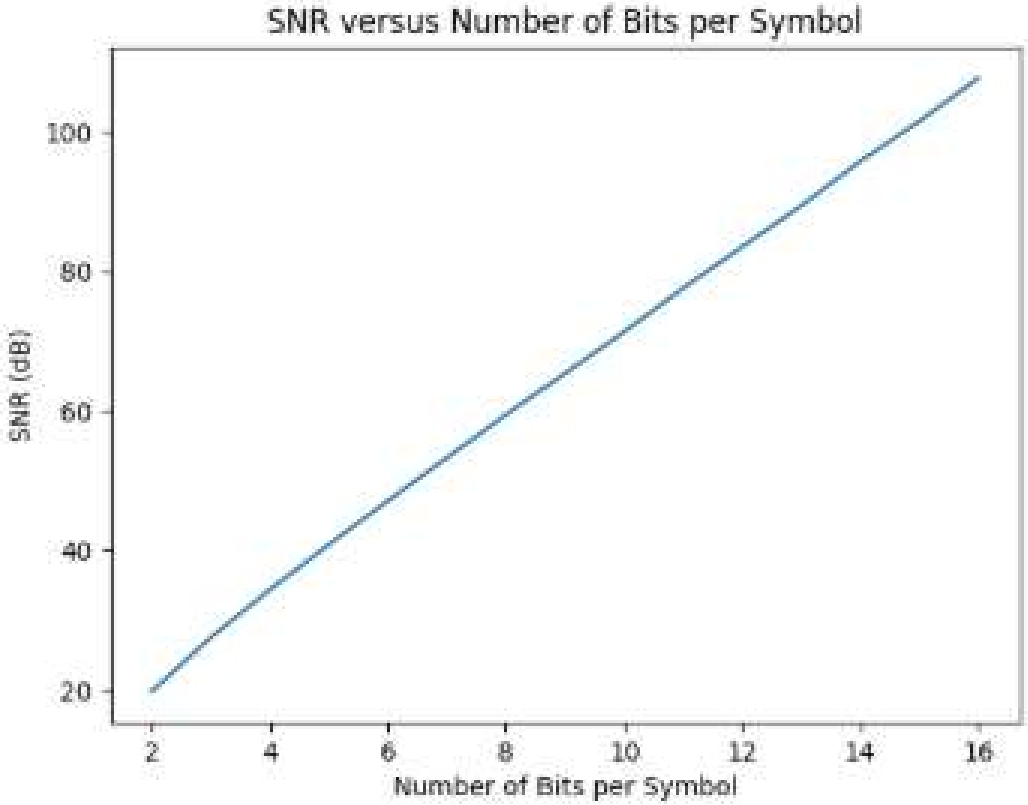
12. Plot the SNR values against the number of bits per symbol.

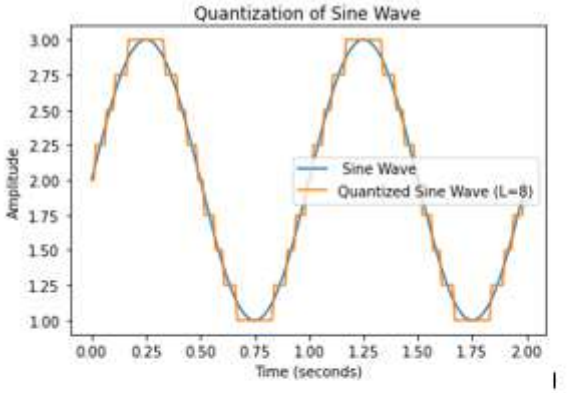
OBSERVATIONS

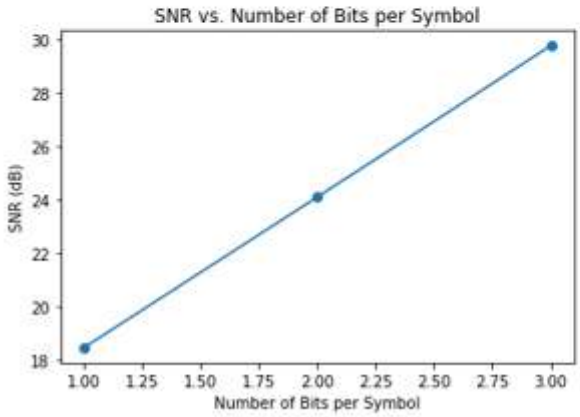












RESULT:

1. Simulated a sinusoidal waveform with dc offset

2. Sampled and quantized the signal using an uniform quantizer and represented each value using decimal to binary encoder

3. Computed SNR in dB

4. Plotted SNR v/s no. of bits

EXPERIMENT - 4

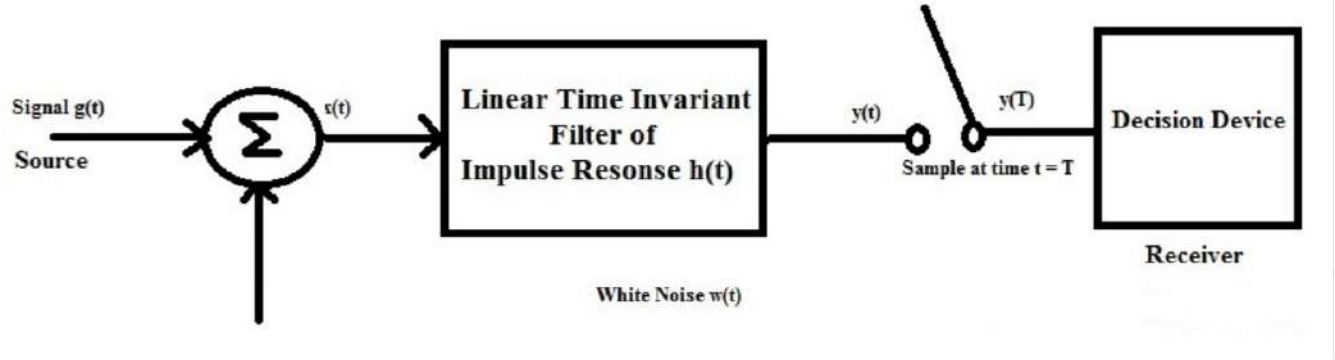
PULSE SHAPING AND MATCHED FILTERING

AIM:

To implement pulse shaping using matched filtering and analyse it.

THEORY:

Matched filter is an optimal linear filter for maximizing SNR in the present of additive stochastic noise . Matched filter are commonly used in radar in which a signal is sent out and we measure the reflected signal looking for something similar to what we sent out. The sinusoidally matched filter are commonly used in image processing . A general representation of matched filter is given below,



The filter input x(t) consist of a pulse signal g(t) corrupted by AWGN as shown by x(t) = g(t) +w(t) 0<=t<=T where T is a arbitrary observation interval . The pulse signal g(t) may represent a binary symbol 1 or 0 in digital communication system. The noise w(t) is the sample function of a white noise porcess is zero mean and power spectral density $\frac{N0}{2}$.

The source of uncertainty lies in the noise w(t). The function of the reciever is to detect the pulse shaping signal in the optimum mannar given the reciever signal x(t) . To satisfy the requirements we have to optimise the design of the filter so as to minimise the offect of noise at the filter output in some statistical sense and thereby enhance the detection of the pulse signal g(t) since the filter is linear the resulting y(t) may be expressed as,

$$y(t) = g_o(t) + h(t)$$

where g(t) and h(t) are produced by the signal noise component of the input x(t) A simple way to describing the requirements that the outptg(t) of the signal be considerably greater athe output noise component . W(t) is to have the filter make the instantaneous power in the output signal g(t) measured at time t =T as large as possible compared to the average power of the output noise n(t)

This is equivalent to maximizing the peak pulse SNR defined as,

$$\eta = \frac{|\ g_o(T)^2\ |}{E(n^2(t))}$$

PYTHON CODE

Root Raised Cosince Filter Code

```
import numpy as np
import matplotlib.pyplot as plt
from commpy import rrcosfilter as rrc
from numpy.random import standard_normal
N=8
L=8
bits=np.random.randint(2,size=N)
plt.figure(1)
plt.stem(bits)
x=np.array([])
for i in bits:
      pulse=np.zeros(L)
      pulse[0]=i*2-1
       x=np.concatenate((x,pulse))
plt.figure(2)
plt.stem(x)
num=101
alpha=0.4
Ts=1
Fs=1
t,h=(num,alpha,Ts,Fs)
y=np.convolve(x,h)
t1=np.arange(0,len(y))
plt.figure(3)
plt.plot(t1,y)
snr=10
s1_power=(sum(abs(y)**2))/len(y)
```

```
SNR=10**(snr/10)
no_power=s1_power/SNR
n=np.sqrt(no_power/2)*standard_normal(y.shape)
r=y+n
plt.figure(4)
plt.plot(r)
r1=np.convolve(r,h)
tr=np.arange(0,len(r1))
plt.figure(5)
plt.plot(tr,r1)
filterdelay=int((len(h)-1)/2)
r2=r1[2*filterdelay+1:tr[-1]-(2*filterdelay):l]
plt.figure(6)
plt.stem(r2)
x_hat=(r2>0)
print(x_hat)
plt.figure(7)
plt.stem(x_hat)
BER1=np.sum(bits-x_hat)
BER=np.sum(bits-x_hat)/N
print('Bit error ',BER1)
```

4 Algorithm

1. Import necessary libraries: ‘numpy‘, ‘matplotlib.pyplot‘, ‘commpy.rrcosfilter‘, and ‘numpy.random.standard$_no$

2. Initialize variables ‘N‘ and ‘L‘ representing the number of bits and the length of each pulse, respectively.

3. Generate random binary bits of length ‘N‘ using ‘np.random.randint‘.

4. Plot the binary bits using ‘plt.stem‘.

5. Iterate through each bit in ‘bits‘:

- Generate a pulse with amplitude determined by the bit value (‘0‘ or ‘1‘) and length ‘L‘.

- Concatenate the pulses to form the modulated signal ‘x‘.

6. Plot the modulated signal ‘x‘ using ‘plt.stem‘.

7. Set parameters for the root-raised cosine filter: ‘num‘, ‘alpha‘, ‘Ts‘, and ‘Fs‘.

8. Generate the root-raised cosine filter ‘h‘ using ‘rrc.rrcosfilter‘.

9. Convolve the modulated signal ‘x‘ with the root-raised cosine filter ‘h‘ to simulate transmission through a communication channel.

10. Plot the transmitted signal ‘y‘.

11. Set the Signal-to-Noise Ratio (SNR) to ‘snr‘.

12. Calculate the signal power ‘s1$_power$‘$ofthetransmittedsignal$‘y‘.

13. Calculate the noise power ‘no$_power$‘$basedontheSNR$.

14. Generate Gaussian noise ‘n‘ with zero mean and calculated noise power.

15. Add noise ‘n‘ to the transmitted signal ‘y‘ to simulate channel noise, resulting in received signal ‘r‘.

16. Plot the received signal ‘r‘.

17. Convolve the received signal ‘r‘ with the root-raised cosine filter ‘h‘ to perform matched filtering.

18. Remove filter delay by discarding initial and final samples.

19. Downsample the filtered signal by a factor of ‘L‘ to obtain the received binary sequence ‘r2‘.

20. Plot the received binary sequence ‘r2‘.

21. Threshold the received binary sequence ‘r2‘ to obtain the decoded binary bits ‘x$_hat$‘.

22. Plot the decoded binary bits ‘x$_hat$‘.

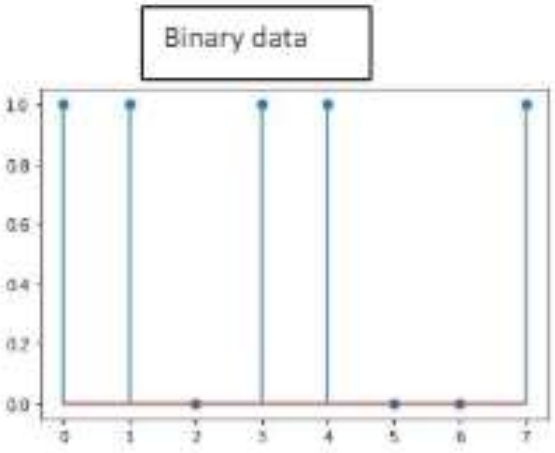
23. Calculate the number of bit errors ‘BER1‘ by comparing the original bits with the decoded bits.

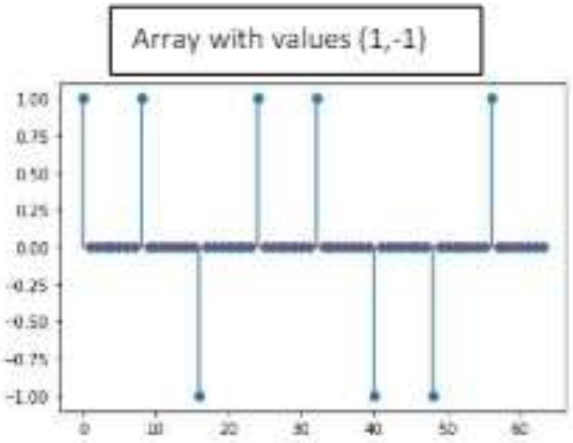
24. Calculate the Bit Error Rate ‘BER‘ by dividing the number of bit errors by the total number of

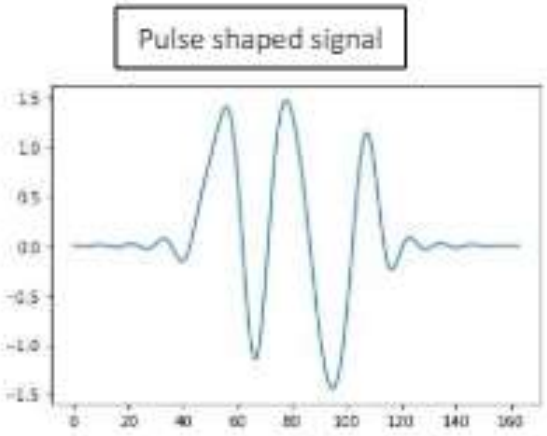
bits.

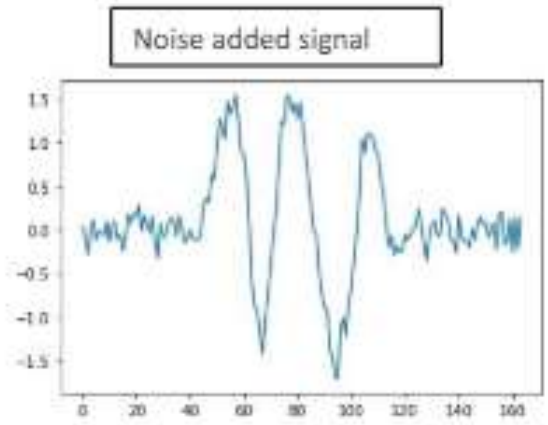
25. Print the number of bit errors and the Bit Error Rate.

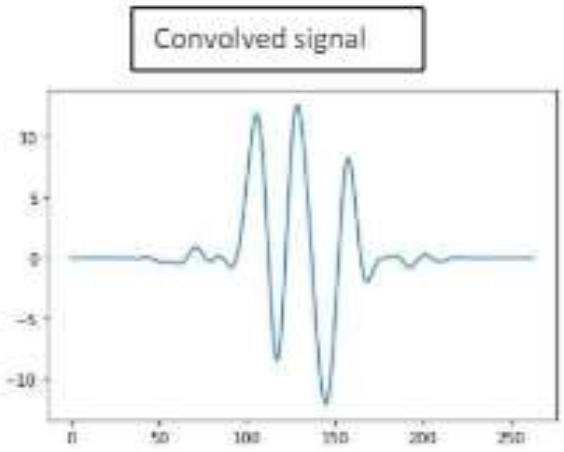
OBSERVATIONS:

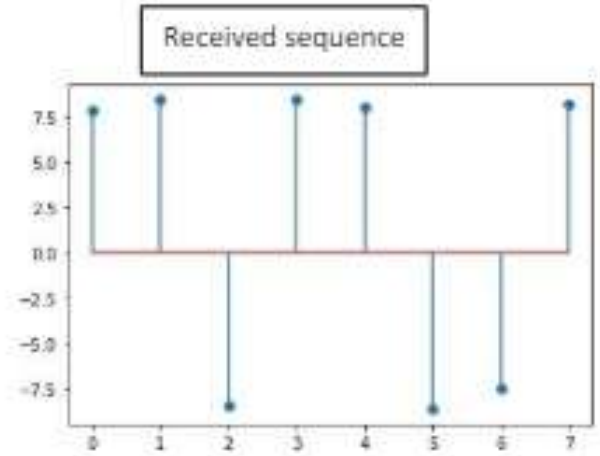












RESULT:

1. Simulated the bitstream.

2. Converted to arrays with values (1,-1) and performed pulse shaping by convoluting with simulated RRC pulse.

3. Added Gaussian noise to the transmitted signal.

4. At the receiver, convolved with simulated RRC pulse for implementing matched filter.

5. Detected received sequence.

6. Calculated BER.

EXPERIMENT - 5

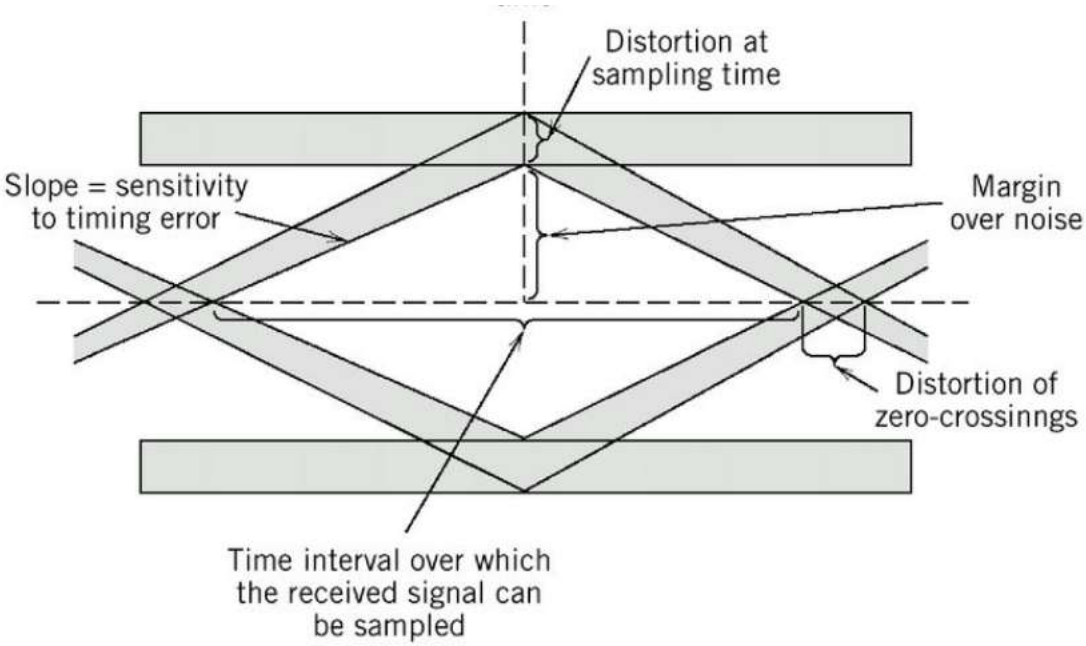
EYE DIAGRAM

AIM:

To generate eye diagram for various roll off factors.

THEORY:

In communication an eye diagram is used to visually access the performance of a system in operation . It is called an eye diagram or eye patteran as it looks like a series of eyes between a pair of rails . It is created by taking the time domain siganl and overlapping the travel for a certain number of symbols if we are sampling a sigbal at a rate of 10 samples per second and want to look at 2 symbols then we would cut the signal every 20 samples and overlap them. In a digital transmission a succession of 1's and 0's follow the reciever . The transmission can consist of long series of zeros . A regular or irregular periodic sequence . A quasi random series or any other combination .The eye diagram will reveal whether everything works as intended or if there are that can affect the transmission causing errors Example : The reception of zero when one is sent.



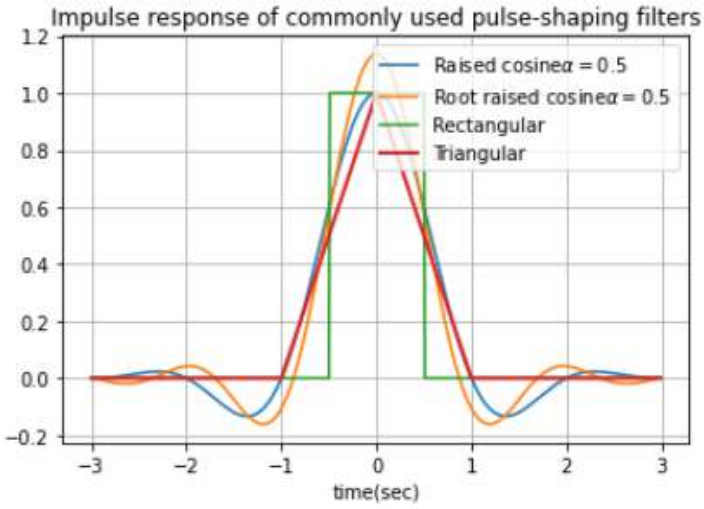
PYTHON CODE

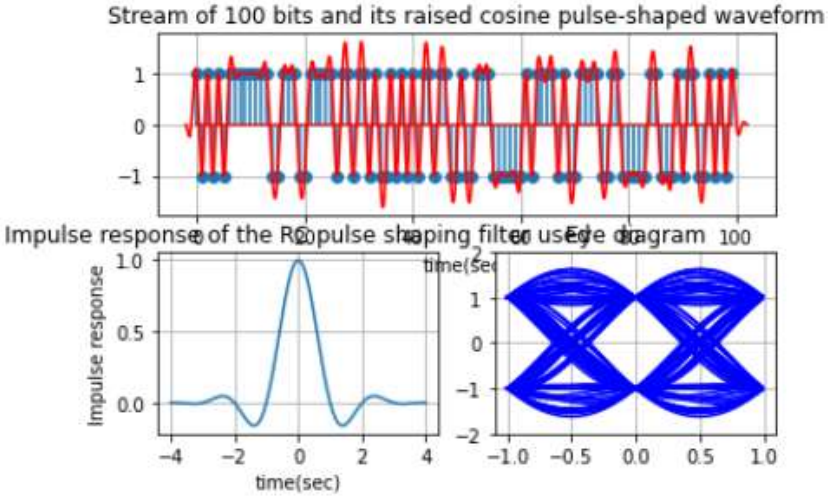
```
import matplotlib.pyplot as plt
import numpy as np


def get_filter(name, T, rolloff = None):
    def rc(t, beta):
```

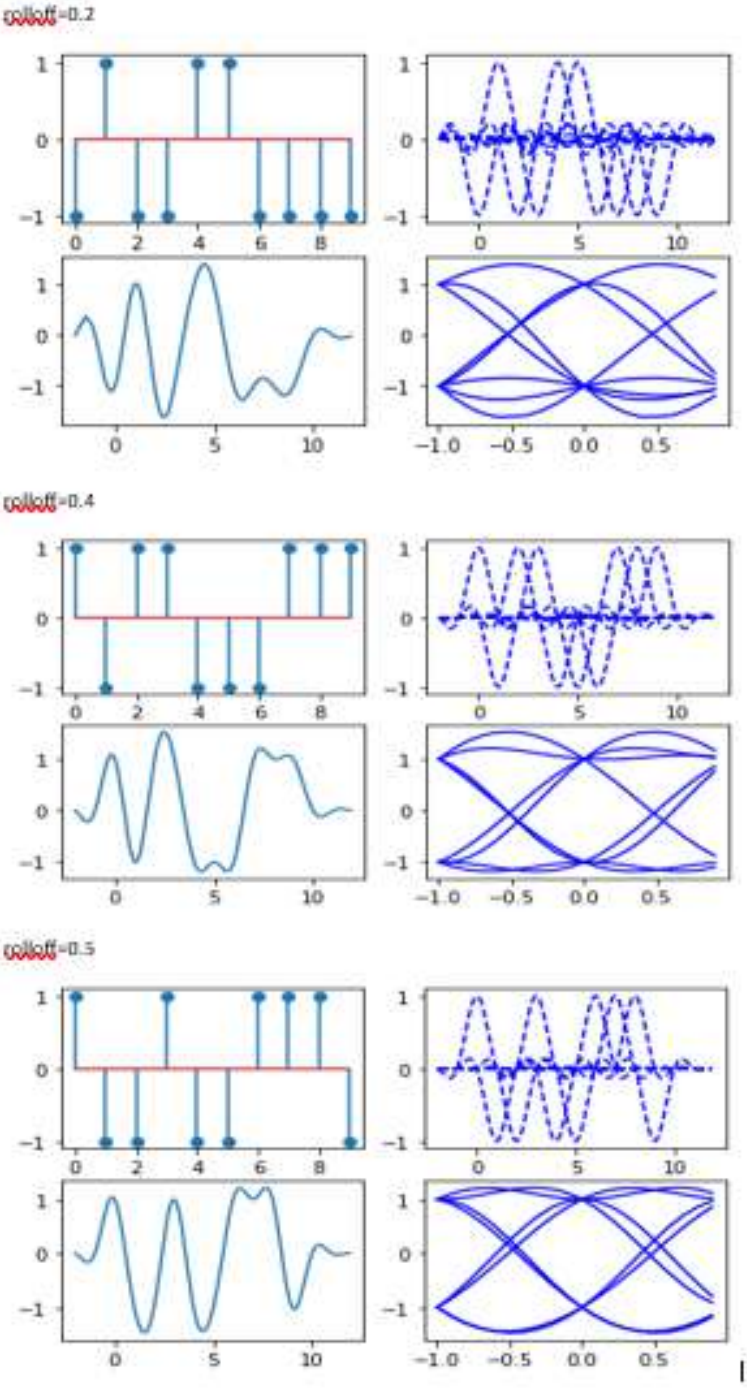
```
        return ((np.sinc(t)*(np.cos(np.pi*beta*t)))/(1-((2*beta*t)**2)))

    def rrc(t, beta):
        return (np.sin(np.pi*t*(1-beta)) + (4*beta*t*np.cos(np.pi*t*(1-
        beta))))/(np.pi*t*(1-((4*beta*t)**2)))

    if name == 'rect':
        return lambda t: (abs(t/T)<0.5).astype(int)
    if name == 'triang':
        return lambda t:(1-abs(t/T))*(abs(t/T)<1).astype(float)
    if name == 'rc':
        return lambda t:rc(t/T, rolloff)
    if name == 'rrc':
        return lambda t:rrc(t/T, rolloff)

def get_signal(g, data):
    t = np.arange(-2*T, (len(data)+2)*(T), 1/Fs)
    xt = sum(data[k] + g(t - k*T) for k in range(len(data)))
    return t, xt

def drawFullEyeDiagram(xt):
    samples_perT = Fs * T
    samples_perWindow = 2*Fs*T
    startInd = 2*samples_perT
    parts = []
    for k in range(int(len(xt)/samples_perT) - 6):
        parts.append(xt[startInd + k * samples_perT + np.arange(samples_perWindow)])
    parts = np.array(parts).T
    t_parts = np.arange(-T, T, 1/Fs)
    plt.plot(t_parts, parts, 'b-')
    plt.show()
```

```
def drawSignals(g, data = None):
    N = 100
    if data is None:
        b = np.random.randint(2, size = N)
        data = 2*b-1
    t, xt = get_signal(g, data)
    t_g = np.arange(-4*T, 4*T, 1/Fs)
    plt.subplot(2,2,3)
    plt.plot(t_g, g(t_g))
    plt.title('SRC Filter')
    plt.subplot(2,1,1)
    plt.stem(data)
    plt.plot(t,xt,'r',label = 'Shaped')
    plt.title('Pulse Shaped')
    plt.subplot(2,2,4)
    drawFullEyeDiagram(xt)
    plt.show()

def showRCEyeDiagram(alpha):
    g = get_filter('rc', T = 1, rolloff = alpha)
    drawSignals(g)
T = 1
Fs = 100
t = np.arange(-3*T, 3*T, 1/Fs)
g = get_filter('rc', T, rolloff = 0.5)
plt.plot(t, get_filter('rect', T)(t), 'c')
plt.plot(t, get_filter('triang', T)(t), 'r')
plt.plot(t, get_filter('rc', T, 0.5)(t), 'y')
plt.plot(t, get_filter('rrc', T, 0.5)(t), 'b')
plt.show()

showRCEyeDiagram(alpha = 0.4)
```

OBSERVATIONS






rolloff=0.2
rolloff=0.4
rolloff=0.5

RESULTS:

Eye diagram for the given roll off factor has been plotted successfully.

Part C

SOFTWARE DEFINED RADIO EXPERIMENTS

EXPERIMENT - 1

FAMILIARISATION OF SOFTWARE DEFINED RADIO

Software-defined radio (SDR)

A software-defined radio (SDR) system is a radio communication system that uses software to process various signals (modulation, demodulation, decoding, etc.) in lieu of the traditional hardware components that are generally made for those dedicated tasks. The advent of modern computing and analog to digital converters allows most of these hardware-based components to be shortlisted in software instead, hence the name software defined radio. This enables cheap signal processing and thus cheap wideband scanner to be produced.

RTL SDR

The RTL-SDR is an ultra-cheap software defined radio based on DVB-T TV tuners with RTL2832U chips. The RTL-SDR can be used as a wide band radio scanner. It may interest ham radio enthusiasts, hardware hackers, tinkerers and anyone interested in RF. The raw data in the RTL2832U chipset could be accessed directly, which allows the DVB-T TV tuner to be converted into a wideband SDR via a custom software driver. The RTL 2832U outputs 8 bit I/Q samples and the highest theoretically possible sample rate is 3.2 Ms/S. But the highest sample rate without loss of samples that have been tested with regular USB controller so far is 2.4Ms/s. The native resolution is 8 bits but the effective no of bits is at -7. Applications of RTL SDR include:

- Use as a police radio scanner.
- Listening to EMS/Ambulance/Fire communications.
- Listening to aircraft traffic control conversations.
- Tracking aircraft positions like a radar with ADSB decoding.
- Decoding aircraft ACARS short messages.
- Scanning trunking radio conversations.
- Radio astronomy.

GNU RADIO

GNU Radio is a free and open-source software development toolkit that provides signal processing blocks to implement software radios. It can be used with readily-available low-cost external RF hardware to create software-defined radios, or without hardware in a simulation-like environment. It is widely used in research, industry, academia, government, and hobbyist environments to support both wireless communications research and real-world radio systems. GNU Radio performs all the signal processing. It can be used to write applications, to receive data out of digital

streams or to push data into digital streams, which is then transmitted using hardware. GNU Radio has filters, channel codes, synchronisation elements, equalizers, demodulators, vocoders, decoders, and many other which are typically found in radio systems. It also includes a method of connecting these blocks and manages how data is passed from one block to another. Extending GNU Radio is also quite easy; if one finds a specific block that is missing, one can quickly create and add it. Since GNU Radio is software, it can only handle digital data. Usually, complex baseband samples are the input data type for receivers and the output data type for transmitters. Analog hardware is then used to shift the signal to the desired centre frequency. GNU Radio applications are primarily written using the Python programming language, while the supplied, performance-critical signal processing path is implemented in C++ using processor floating point extensions, where available. Thus, the developer is able to implement real-time, high-throughput radio systems in a simple-to-use, rapid-application-development environment.

Signal Generation, Plotting Signal Spectrum and Filtering of Signals

AIM:

To familiarise available blocks in GNU Radio and to study how signals are generated, spectrum of signals(or power spectral density) can be analysed and also study how filtering can be performed.

COMPONENTS REQUIRED:

Software required: GNU radio software

Hardware required: Antenna

THEORY

RTL-SDR dongle provides raw data to GNU Radio. All signal processing is performed using reconfigurable software. That is, GNU Radio is used for implementing the receiver by selecting signal processing blocks, which process the incoming signal from RTL-SDR dongle to demodulate FM signal and to get audio output from the computer speakers. The antenna is a standard antenna provided with RTL-SDR dongle with a magnetic base, as shown in Fig. 5. It provides acceptable FM reception in frequency range of 88MHz to 108MHz. Place the antenna at a proper place.

Steps for designing FM receiver in GNU Radio

Generate, execute and kill are three important tools in GNU Radio software. The program flowgraph consists of various blocks

Signal source

Signal Source in GNU radio generates a variety of signals for signal processing, modulation, simulation and visualization purposes. A sample rate of 48KHz in signal source matches the standard auto

sampling rate while setting waveform as a cosine with frequency 1kHz and offset 0 generates a 1kHz continuous wave signal with no DC offset

Multiply

This block is used to perform element wise multiplication of two imput signals, commonly used for modulation operations.

QI GUI time sink

The block displays the time domain waveform of a signal, providing real time visualization of its amplitude variations at a sample rate of 48kHz with a fixed amplitude scale for consistent visualisation.

QI GUI frequency sink

The QI GUI frequency sink block visualises the frequency spectrum of a signal in real time seeing the no of points as 1024 in QT GUI interface. The frequency sink displays a frequency spectrum with 1024 points centered around 0 Hz and spanning a bandwidth of 48KHz

High Pass filter

This block removes low frequency components from a signal, emphasising higher frequency content and filtering out unwanted lower frequency noise. In HPF, a decimation value of 1 means that the output sampling rate is the same as the input sample rate. Gain of 1 ensures that no amplification or attenuation with 5K as a frequency below which a signal is attenuated. A transition width of 500 specifies the range over which filter transitions from pass band to stopband and a beta value of 6.76 indicates the parameter for shaping filter response.

Complex to Real

This block converts complex valued signal into real valued signals by extracting the real component, enabling processing in the real domain.

Audio Sink

Plays back audio signals in real time.

PROCEDURE:

1. Launch GNU Radio
2. Open GNU Radio Companion
3. Create a new flowgraph
4. Add signal source, high pass filter and audio sink blocks.
5. Connect the blocks. Headphones is connected as output audio device.
6. Configure the blocks.

7. Save the flowgraphs.

8. Execute. The graphical sink block will display generated signal.

9. Listen to the audio output.

10. Modify and iterate for improvements or exploration. Spectrum of signals generated and filtered signals will be displayed in the time domain.

RESULT:

The signal generation and spectrum of signals is analysed. Also studied how filtering of signals is performed using GNU radio software.

EXPERIMENT - 2

FM RECEPTION

AIM:

To receive the digitalised FM signal using SDR board and setup an LPF and FM receiver using GNU Radio.

COMPONENTS REQUIRED:

GNU radio software, antennae, RTL SDR dongle with RTL chip, headphones.

THEORY

Block details

The antenna is a standard antenna provided with RTL-SDR dongle with a magnetic base, as shown in Fig. 5. It provides acceptable FM reception in frequency range of 88MHz to 108MHz. Place the antenna at a proper place. Generate, execute and kill are three important tools in GNU Radio software. The program flowgraph consists of various blocks in signal generation.

Soapy RTL SDR Source

It provides a hardware abstraction for many software defined radio devices. Soapy RTL SDR is a module that allows GNU radio to interface with RTL SDR dongle using the soapy SDR library. This source slock is used in GNU radio to interact with RTL SDR devices through the soapy SDR interface. The sample rate by default is 2.16 Ms/s. It determines the rate at which the dongle captures digital sample of the received signal.

Low pass filter

This block removes high frequency components from a signal, emphasising lower frequency content and filtering out unwanted higher frequency noise. They are commonly used to remove noise, extract baseband signals or prepare signal for modulation or demodulation, Parameters such as sample rate, cutoff frequencies and signal length can be adjusted to control the filtering behaviour. In LPF, a decimation value of 1 means that there is no downsampling or reduction in sample rate. Here frequencies above 100 KHz are attenuated while those below it are allowed to pass. A transition width of 50 KHz specifies the range over which filter transitions from pass band to stopband. Hamming window with a beta value of 6.76 indicates the window function and shape parameter.

QI GUI time sink

The block displays the time domain waveform of a signal, providing real time visualization of its amplitude variations at a sample rate of 2.16M Hz with a fixed amplitude scale for consistent visualisation.

QI GUI frequency sink

The QI GUI frequency sink block visualises the frequency spectrum of a signal in real time seeing the number of points as 1024 in QT GUI interface. Setting a FFT size as 1024 in GUI frequency sink allows for higher frequencies in the displayed spectrum, allowing more detailed analysis of individual components in the input. The entire available spectrum is centered around 0 Hz and spanning a bandwidth of 2.16MHz in wideband configuration.

WBFM Receiver

Wideband frequency modulation is to capture and demodulate WBFM signals. Quadrature rate of 48KHz is chosen to match the desired audio sample for demodulating.

Audio Sink

Plays back audio signals in real time through the computer's audio output device.

PROCEDURE:

1. Launch GNU radio companion

2. Drag and drop a signal source from the source category to the GRC canvas. Also drop a LPF, QT GUI frequency sink and audio sink.

3. Double clock on the blocks to configure its parameters such as amplitude, frequency and waveform type.

4. Connect the output of the signal source block to a graphical sink block such as time sink or GUI sink.

5. Save the GRC flowgraph and click execute button to run.

6. Listen to the audio output through the headphones connected.

RESULT:

Demodulated the output of FM radio station tuned to a specify frequency and was able to listen to the audio signal from the FM station through the audio output device connected to the computer.